

El CRUD del DAO

Esta clase fue el profesor escribiendo `CarreraDAO` completo en el pizarrón, de principio a fin. Arrancó donde quedó la clase 3 (las constantes SQL y la conexión) y terminó con los cinco métodos, la clase `Principal` que los usa, y un stored procedure de pilón.

Escribió cada método en un color distinto de marcador, y así está organizada esta página: azul el `create`, verde el `update`, rojo el `delete`, negro el `read`. Entre bloque y bloque hay **nueve ejercicios** para que no sea nada más copiar el pizarrón, y las **cuatro tareas** aparecen justo en el minuto en que las dijo.

Los botones con ? explican los términos que dio por sabidos. Ábrelos cuando algo no te suene; no interrumpen la lectura.

DÓNDE NOS QUEDAMOS · MINUTO 00:32

El punto de partida

Ni él se acordaba del nombre de la clase: "recuérdense, ¿la clase era cursos, materias...?" — "Carrera", le contestaron. Vamos a seguir jugando con ese.

De la clase 3 traemos dos cosas ya escritas: la clase `Carrera` (tres atributos privados, constructor vacío, getters, setters y `toString`) y el esqueleto de `CarreraDAO` con el método que se conecta a la base de datos. Todo lo de hoy se cuelga de ahí.

— ¿Por qué la clase se llama `CarreraDAO` y no nada más `Carrera` ?

Son dos clases con trabajos distintos, y por eso son dos archivos distintos:

- `Carrera` es el **dato**. Solo guarda un id, un nombre y una descripción. No sabe que existe una base de datos.
- `CarreraDAO` es el **que habla con la base de datos**. Sabe conectarse, sabe SQL, y sabe convertir filas de tabla en objetos `Carrera` y al revés.

Esa separación es todo el **patrón DAO** (Data Access Object): si mañana cambias de MySQL a PostgreSQL, tocas `CarreraDAO` y nada más. La clase `Carrera` ni se entera, y el resto de tu aplicación tampoco.

El sufijo `DAO` en el nombre es puro convenio para que se lea de inmediato qué hace. Como él lo dijo: *"solo por el gusto de ponerle un identificador para tratar de dar a*

*entender que vamos a escribir una variante del patrón de diseño". Ojo con esa palabra: **variante**. El patrón formal pide una interfaz más una implementación; aquí se está escribiendo la versión corta.*

BLOQUE 1 · MINUTO 00:48 A 07:33

Las cinco constantes SQL

Antes de cualquier método, arriba de la clase van las consultas guardadas como constantes. Una por operación del CRUD.

La forma es siempre la misma y la escribió cinco veces:

```
// las cinco, arriba de todo, dentro de la clase CarreraDAO
private static final String SQL_INSERT =
    "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)";

private static final String SQL_UPDATE =
    "UPDATE Carrera SET nombreCarrera = ?, descripcionCarrera = ? WHERE idCarrera = ?";

private static final String SQL_DELETE =
    "DELETE FROM Carrera WHERE idCarrera = ?";

private static final String SQL_SELECT =
    "SELECT * FROM Carrera WHERE idCarrera = ?";

private static final String SQL_SELECT_ALL =
    "SELECT * FROM Carrera";
```

– Desármame `private static final String`, palabra por palabra

Son cuatro cosas apiladas y cada una dice algo distinto:

- `private` — solo el código dentro de `CarreraDAO` puede verla. Nadie de fuera necesita saber qué SQL usas.
- `static` — **existe una sola copia, compartida por toda la clase**, no una por cada objeto. Si creas cien `CarreraDAO`, el texto del INSERT sigue estando una sola vez en memoria. Tiene sentido porque la consulta nunca cambia entre un objeto y otro.
- `final` — una vez asignada, no se puede reasignar. Intentar `SQL_INSERT = "otra cosa"` más adelante no compila.
- `String` — el tipo. Es texto, nada más. Java no sabe ni le importa que ese texto sea SQL; para él es una cadena cualquiera.

Y sobre el nombre en **MAYÚSCULAS con guiones bajos**: eso no es obligación del compilador, es convención. Él lo dijo así: *"eso más que el nombre del atributo que esté en mayúsculas, se entiende que es una constante"*. En Java **no existe la palabra** `const` —

está reservada pero no se puede usar. `private static final` más el nombre en mayúsculas es lo más cercano que hay.

— ¿Qué hacen los signos `?` ahí sueltos?

Son **huecos**. La consulta queda escrita completa salvo por los valores, que se rellenan justo antes de ejecutarla.

Esto es lo que hace un `PreparedStatement`: manda la consulta al motor de base de datos *con los huecos vacíos*, el motor la analiza y la deja lista, y hasta después le pasas los valores por separado. Como el motor ya decidió qué es estructura y qué es dato, un valor malicioso no puede convertirse en instrucción.

Ese es el motivo real de que en la clase 3 dijera que **nunca usen** `Statement`: con `Statement` concatenas el valor dentro del texto SQL, y ahí es donde vive la inyección SQL.

```
// NO. Esto es concatenar: el valor entra al texto de la consulta.  
"SELECT * FROM Carrera WHERE idCarrera = " + id  
  
// SÍ. El hueco viaja aparte del valor.  
"SELECT * FROM Carrera WHERE idCarrera = ?"
```

Sobre escribirlas en una sola línea. Insistió en eso: *"todo eso en una línea, porque si está en múltiples líneas, pues no se les olvide cerrar por acá, concatenar y demás cosas"*. Java no tiene cadenas de varias líneas en la sintaxis clásica: si partes el texto, tienes que cerrar comillas, poner `+`, abrir comillas otra vez, y **acordarte del espacio**. Ese espacio olvidado es el bug clásico:

```
// se convierte en "...FROM CarreraWHERE idCarrera = ?" -> error de sintaxis SQL  
"SELECT * FROM Carrera" +  
"WHERE idCarrera = ?"
```

También mencionó que si necesitas más consultas —*"hacer joins, inner joins, lo que se les ocurra, left, outer"*— aquí es donde van: una constante más y un método más. El DAO crece por ahí, no por otro lado.

EJERCICIO 1

¿Cuántos parámetros recibe cada consulta?

Cuenta los `?`. Ese número es exactamente cuántas líneas `ps.set...`
`()` vas a escribir después, así que conviene que salga automático. **Elige el número para cada una.**

SQL_INSERT

INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)

0

1

2

3

SQL_UPDATE

UPDATE Carrera SET nombreCarrera = ?, descripcionCarrera = ? WHERE idCarrera = ?

1

2

3

4

SQL_DELETE

DELETE FROM Carrera WHERE idCarrera = ?

0

1

2

3

SQL_SELECT

SELECT * FROM Carrera WHERE idCarrera = ?

0

1

2

3

SQL_SELECT_ALL

SELECT * FROM Carrera

0

1

2

3

Revisar

Reiniciar

DIGRESIÓN · MINUTO 03:50 A 05:37

"Esta cosa no se debe de hacer"

Escribió el `DELETE` y de inmediato se frenó a decir que en la vida real no lo van a usar.

"Solo para fines didácticos. Es decir, para cualquier materia que tengan en la escuela o cuando estén jugando. Esta cosa no se debe de hacer. Ustedes no deberían de eliminar nada por normatividad vigente."

Un alumno contestó que la retención depende del giro, "de 5 a 10 años". El profesor corrigió: la ley de transparencia y acceso a la información marca 12. Y añadió el matiz que importa: aunque tu giro pida 5, si el último día del año 5 llega una auditoría pidiendo 5 años más, estás obligado a entregar. **Aplica igual para documentos físicos y electrónicos.**

Traducido a código: el `DELETE FROM` literal borra la fila y ya no queda nada que auditar. Por eso lo que sigue es tarea.

TAREA 1 minuto 05:37

Investigar el *soft delete*

"Si bien es cierto esto es para fines didácticos, lo vamos a trabajar así; para fines del proyecto hay un concepto que se llama soft delete. Averigüen qué es, con qué se come, cómo se adereza y cómo lo aplican."

El formato ya lo conoces. Esas cuatro preguntas son siempre las mismas con él —*qué es, para qué sirve, con qué se come y cómo se adereza*— y son las que usó para DTO y DAO en la clase 3. Contéstalas en ese orden y agrega la cuarta que añadió aquí: **cómo lo aplican.**

La pista ya salió en clase. Un alumno preguntó "¿tiene que ver con los grados de borrado, borrado suave y borrado duro?" y él confirmó: *"sí, justo. O borrado lógico, borrado físico"*.

Es para el proyecto, no solo para entregar. Dijo textualmente "para fines del proyecto". Si tu equipo va a tener tablas con datos que se dan de baja, esto define cómo diseñas el esquema desde ahorita.

– Adelanto: la idea del borrado lógico en una línea

En vez de sacar la fila de la tabla, le pones una marca de que ya no está vigente. Una columna extra, típicamente `activo BOOLEAN` o `fechaBaja DATETIME`.

El `DELETE` deja de ser un `DELETE` y se convierte en un `UPDATE`:

```
-- borrado físico: la fila desaparece
DELETE FROM Carrera WHERE idCarrera = ?;

-- borrado lógico: la fila sigue ahí, marcada
UPDATE Carrera SET activo = false WHERE idCarrera = ?;
```

El costo es que **todas** tus consultas de lectura tienen que empezar a filtrar (`WHERE activo = true`), y si se te olvida en una, aparecen registros fantasma. Ese trade-off es justo lo que la investigación tiene que discutir: no te quedes en la definición.

Un cabo que amarra con la clase 3: en el DTO de inserción, `activo` normalmente **no va**, porque la base de datos ya le pone su valor por defecto.

BLOQUE 2 · MINUTO 07:56 A 19:43

create() y la anatomía de un método DAO

El primer método completo. Todos los demás son variaciones de esta misma plantilla, así que aquí es donde conviene entender de verdad.

Antes de los métodos hay que declarar un atributo de la clase que guarde la conexión viva. Él lo puso a media pizarra buscando espacio: `Connection conexion`; . Con eso, el método queda:

```
public void create(Carrera c) throws SQLException {  
  
    obtenerConexion();                // 1. llena el atributo conexion  
    PreparedStatement ps = null;       // 2. lo declara vacío  
  
    try {  
        ps = conexion.prepareStatement(SQL_INSERT); // 3. prepara la consulta  
        ps.setString(1, c.getNombreCarrera());    // 4. llena el hueco 1  
        ps.setString(2, c.getDescripcionCarrera()); // y el hueco 2  
        ps.executeUpdate();                       // 5. la ejecuta  
    } finally {  
        if (ps != null) ps.close();               // 6. cierra, pase lo que pase  
        if (conexion != null) conexion.close();  
    }  
}
```

Seis pasos. Los cinco métodos del CRUD son este mismo esqueleto cambiando la constante, los `set` y la forma de ejecutar. Vamos uno por uno.

Pasos 1 y 2 · La conexión y el PreparedStatement

La primera línea del método llama al método de conexión que escribieron la clase pasada. Ni él se acordaba del nombre exacto: *"si el método que escribimos en esa ocasión se llama `conéctate()`, pues `conéctate()`".* Ahí adentro está el `DriverManager.getConnection()` cuyo resultado se guarda en el atributo `conexion`.

— ¿Qué es exactamente un `PreparedStatement` ?

Es el objeto de Java que representa **una consulta lista para ejecutarse** contra una conexión concreta.

La conexión sola no ejecuta nada: es apenas el cable hacia la base. El `PreparedStatement` es lo que viaja por ese cable. El flujo completo es:

1. Le pides a la conexión que prepare un texto SQL:

```
conexion.prepareStatement(SQL_INSERT) .
```

2. Te devuelve un objeto `PreparedStatement`. La base ya analizó la consulta y sabe qué va a hacer.

3. Le pasas los valores de los huecos con `setString`, `setInt`, etcétera.

4. Le dices que se ejecute.

Se declara en `null` **afuera** del `try` a propósito. Si lo declararas adentro, el `finally` no podría verlo para cerrarlo: una variable solo vive dentro de las llaves donde se declaró.

Los tres verbos que usó para describir `prepareStatement` son buenos: *"¿qué consulta va a parametrizar, va a programar, va a planificar?"*.

Paso 4 · Los índices empiezan en 1

Aquí está la parte que más se equivoca. Numeró los `?` en el pizarrón y luego numeró los `set`:

"La lista de argumentos empieza en 1 y termina en N."

No en cero. Es una de las poquísimas cosas en Java que no arranca en cero, y es precisamente porque ese índice no es de Java: es del SQL. La firma completa es `ps.setString(índice, valor)`: primero **cuál** hueco, después **qué** le metes.

¿Y de dónde sale el valor? Del objeto `Carrera` que llegó como argumento. Preguntó en clase quién conoce el nombre de la carrera, y la respuesta es el propio objeto: `c.getNombreCarrera()`. Eso implica que **alguien tuvo que llenar ese objeto antes**, en otro lado:

```
Carrera c = new Carrera();
c.setNombreCarrera("ISC");
c.setDescripcionCarrera("La carrera de ISC");

// ahora c "conoce" esos datos, y es lo que se le pasa al DAO
dao.create(c);
```

EJERCICIO 2

Empareja cada hueco con su setter

Esta es la consulta del `UPDATE`, la que tiene los tres huecos:

```
UPDATE Carrera
  SET nombreCarrera = ?, descripcionCarrera = ?
 WHERE idCarrera = ?
```

¿Qué línea corresponde a cada hueco? Fíjate en el índice y en el tipo al mismo tiempo: hay opciones que fallan en uno y aciertan en el otro.

Hueco 1 (nombreCarrera = ?)

`ps.setString(1, c.getNombreCarrera())`

`ps.setString(0, c.getNombreCarrera())`

`ps.setInt(1, c.getNombreCarrera())`

Hueco 2 (descripcionCarrera = ?)

`ps.setString(1, c.getDescripcionCarrera())`

`ps.setString(2, c.getDescripcionCarrera())`

`ps.setString(2, c.getNombreCarrera())`

Hueco 3 (WHERE idCarrera = ?)

`ps.setString(3, c.getIdCarrera())`

`ps.setInt(1, c.getIdCarrera())`

`ps.setInt(3, c.getIdCarrera())`

Revisar

Reiniciar

El tipo del setter tiene que coincidir con el de la columna

Se detuvo un buen rato aquí, aunque en `Carrera` no se note porque los dos campos son texto —*"no es tan complejo: es copiar y pegar"*—. Así que se inventó un caso: supón que en lugar de `descripcion` tuvieras `duracion`, y fuera un entero, los años que dura la carrera.

"Que lo que nosotros pasemos como argumento uno coincida con el tipo de dato que estamos pasando. Si este es un VARCHAR, aquí tiene que ser un String. Si duración es un entero, aquí tendría que ser un int."

EJERCICIO 3

¿Qué setter le toca a cada columna?

La regla es mecánica una vez que la ves: **el tipo de la columna manda**.
Elige el método correcto para cada una.

nombreCarrera VARCHAR(150)

setString

setInt

setDouble

setBoolean

idCarrera INT

setString

setInt

setDouble

setBoolean

duracion INT — los años que dura la carrera

setString

setInt

setDouble

setBoolean

duracion DOUBLE — porque hay carreras de 4.5 años

setString

setInt

setDouble

setBoolean

activo BOOLEAN — la columna del soft delete

setString

setInt

setDouble

setBoolean

Revisar

Reiniciar

Paso 5 · Solo hay dos formas de ejecutar

Preguntó qué tipos de consulta SQL les enseñaron en Bases de Datos, y fueron saliendo: inserción, actualización, eliminación, selección. Cuatro. Y entonces vino el remate:

"En teoría les presentaron al menos cuatro. Para nosotros solo hay dos:

`ps.executeUpdate()` *y san se acabó."*

Es la simplificación más útil de toda la clase. Desde JDBC, las cuatro familias de SQL colapsan en dos métodos según **qué te devuelven**:

MÉTODO	PARA QUÉ SQL	QUÉ DEVUELVE
<code>executeUpdate()</code>	INSERT, UPDATE, DELETE — todo lo que modifica filas	un <code>int</code> : cuántas filas cambió
<code>executeQuery()</code>	SELECT, con o sin joins	un <code>ResultSet</code> : las filas encontradas

El nombre `executeUpdate` confunde porque suena a que solo sirve para `UPDATE`. No: sirve para las tres que modifican. Piénsalo como "ejecutar una actualización *de la tabla*", no "ejecutar un UPDATE".

EJERCICIO 4

executeUpdate o executeQuery

Seis consultas. **Decide con cuál se ejecuta cada una.** Metí un par que él solo mencionó de pasada ("joins, inner joins, lo que se les ocurra") justo para ver si te quedó la regla o solo memorizaste las cuatro básicas.

INSERT INTO Carrera (nombreCarrera) VALUES (?)

`executeUpdate`

`executeQuery`

SELECT * FROM Carrera

`executeUpdate`

`executeQuery`

DELETE FROM Carrera WHERE idCarrera = ?

`executeUpdate`

`executeQuery`

SELECT c.nombreCarrera, a.nombre FROM Carrera c INNER JOIN Alumno a ON a.idCarrera = c.idCarrera

`executeUpdate`

`executeQuery`

UPDATE Carrera SET activo = false WHERE idCarrera = ?

`executeUpdate`

`executeQuery`

SELECT COUNT(*) FROM Carrera

`executeUpdate`

`executeQuery`

Revisar

Reiniciar

Paso 6 · finally, no catch

Aquí hizo algo que parece descuido y es decisión:

"Como todo bloque try, si intenta hacer algo y si no puedes lanza la excepción, debiera de haber un catch. No le vamos a poner catch, le vamos a poner un finally."

– try / catch / finally: qué hace cada uno

- `try` — "intenta esto". Si algo truena adentro, el resto del bloque se salta.
- `catch` — "si tronó, haz esto en su lugar". **Atrapa el error y lo detiene ahí.**
- `finally` — "esto se ejecuta pase lo que pase". Salga bien, truene, o incluso si hay un `return` en medio.

Su formulación: *"el 'intenta', si no puedes lanza el error, y el otro es 'no me interesa si pudiste o no pudiste, esto se ejecuta'"*.

Poner `finally` **sin** `catch` es legal, y es exactamente lo que quieres aquí: no vas a resolver el error (no es tu trabajo), pero **sí** tienes que cerrar la conexión antes de irte. Si pusieras un `catch`, te tragarías el error y quien llamó al método creería que todo salió bien.

Y adentro del `finally`, las dos preguntas. Primero el statement, después la conexión:

```
} finally {  
    if (ps != null) ps.close();  
    if (conexion != null) conexion.close();  
}
```

– ¿Por qué el `if (ps != null)` ? ¿Y por qué en ese orden?

El **if**: el `finally` se ejecuta siempre, incluso si la conexión falló y `ps` se quedó en `null`. Llamar `.close()` sobre `null` revienta con un `NullPointerException`, y lo peor es que ese error nuevo **tapa** el error original que sí querías ver.

El **orden**: al revés de como se abrieron. Primero se abrió la conexión, luego el statement; se cierran statement primero, conexión al último. El statement depende de la conexión, así que cerrar la conexión antes lo dejaría colgado.

Cerrar importa de verdad: cada conexión abierta ocupa un recurso del servidor de base de datos, y hay un límite. Un método que se olvida de cerrar cuando hay error va agotando el pozo hasta que la aplicación entera deja de poder conectarse. Eso se llama **fuga de conexiones** y es de los bugs más difíciles de rastrear, porque tarda horas en manifestarse.

Las excepciones son una carrera de relevos

Si el método no atrapa el error, ¿quién lo atrapa? Esta fue la mejor analogía de la clase.

"¿Qué pasa con los errores? 'Ah, bueno, es que no pude insertar'. Se lanza una excepción. ¿Quién le va a dar tratamiento? La capa de arriba. Imagínense que están jugando carreras de relevos. ¿Aquí le tiran la estafeta a quién? Pues al que sigue."

El `throws SQLException` en la firma del método es precisamente eso: el aviso de que este método **no** resuelve el problema, solo lo pasa. Y quién es "el que sigue" depende del tipo de aplicación:

TIPO DE APLICACIÓN	QUIÉN RECIBE LA ESTAFETA
Escritorio	La capa que genera la interfaz gráfica: Swing, JavaFX
Web	El Servlet, el JSP — y en el stack del curso, el controlador de Spring
Móvil	La capa que comunica y pinta el error al cliente

— ¿Qué es una "excepción" y qué significa `throws` en la firma?

Ya lo había dicho en la clase 3: *en Java no hay errores, hay excepciones*. Una excepción es un objeto que representa "algo salió mal", que interrumpe el flujo normal y va subiendo por la cadena de quién llamó a quién hasta que alguien la atrape.

`SQLException` es una excepción **checked**: el compilador te *obliga* a hacer algo con ella. Solo tienes dos opciones y no hay tercera:

- **Atraparla** aquí mismo con `try/catch`.
- **Declararla** con `throws SQLException` en la firma, que es decir "yo no, que la atrape el que me llamó".

El DAO elige la segunda. Por eso, en cuanto escribas `dao.create(c)` en otro lado, el compilador te va a marcar error hasta que *ahí* pongas el `try/catch`. Él lo describió como si el compilador te reclamara: *"oye, espérate, el método que tú pretendes ejecutar dice que probablemente genere una excepción. No lo estás tratando allá, lo tengo que tratar yo"*.

EJERCICIO 5

¿Quién trata el error?

Cuatro preguntas sobre el reparto de responsabilidades. **Elige una opción en cada una.**

El método `create()` del DAO lleva `throws SQLException`. ¿Qué significa eso?

Que atrapa el error

Que lo pasa a quien lo llamó

Que lo ignora

El DAO usa `finally` sin `catch`. ¿Por qué?

Es un descuido del pizarrón

Para cerrar la conexión sin tragarse el error

Porque `catch` y `finally` no pueden ir juntos

Es una aplicación web con Spring. ¿Quién recibe la estafeta?

El Servlet o el controlador

La clase Carrera

La base de datos

Ya atrapaste la excepción en la capa de arriba. ¿Qué NO deberías poner?

Un log con SLF4J

`e.getMessage()` en un mensaje

`e.printStackTrace()`

Revisar

Reiniciar

Y cuando por fin la atrapas, no pongas `printStackTrace`

En el `main`, el `catch` más directo sería `catch (SQLException e)`, o el genérico `catch (Exception e)`. Y adentro, lo que todo mundo escribe: `e.printStackTrace()`. Sobre eso fue tajante:

"La realidad dicta que el `printStackTrace()` no lo pongan. Ahí tendrían que poner un log en SLF4J, que es un Simple Logging Facade for Java. O el manejador de logs específico, para que eso lo mande al server y no sea tan escandaloso para el usuario."

– Stack trace, log y SLF4J: qué es cada cosa

Un **stack trace** es la lista de métodos que estaban en curso cuando reventó, del más profundo al más superficial. Es utilísimo para depurar y es exactamente lo que **no** quieres que vea un usuario final: le enseña nombres de tus clases, rutas de archivos y a veces datos.

`printStackTrace()` lo escupe en la consola. Problemas: en un servidor no hay una consola que alguien esté mirando, no queda registrado en ningún lado con fecha, y no

puedes apagarlo ni filtrarlo.

Un **log** es un registro con fecha, nivel de severidad (ERROR, WARN, INFO, DEBUG) y destino configurable: archivo, servidor central, lo que sea. Puedes decir "en producción solo quiero ERROR" sin tocar el código.

SLF4J (Simple Logging Facade for Java) es una *fachada*: una capa intermedia con la que tu código habla, para que la librería de logging real (Logback, Log4j) se pueda cambiar sin tocar tus clases. Es el mismo razonamiento del patrón DAO: hablar con una abstracción, no con la implementación.

Y el detalle que agregé al final: `printStackTrace()` no es lo único que trae la excepción; también está `e.getMessage()`, que da el mensaje específico del error y suele ser lo que sí quieres mostrar o registrar.

PREGUNTA DE CLASE · MINUTO 23:14 A 25:34

"¿Para qué sirven los constructores vacíos?"

Un alumno la lanzó como "pregunta de cultura general" y se llevó dos minutos de respuesta. Vale la pena porque toca algo que ya venían usando sin pensar.

La respuesta corta fue "para inicializar valores", pero la explicación larga distingue tres momentos que se confunden todo el tiempo:

MOMENTO	CON PRIMITIVOS	CON OBJETOS
Declarar	<code>int x;</code>	<code>Carrera c;</code>
Crear	no aplica, ya existe	<code>new Carrera()</code>
Inicializar	<code>x = 10;</code>	<code>c.setNombreCarrera("ISC")</code>

Su explicación del `new`: *"okay, va, mira, hay una plantilla por ahí que se llama el objeto Carrera, tiene estos atributos... bueno, pues créame una instancia de ese objeto"*. La clase es la plantilla; el `new` fabrica un ejemplar.

Sobre qué se puede meter en un constructor vacío, dio dos usos concretos: asignar valores por defecto, y crear instancias de otras clases que el objeto vaya a necesitar. Y remató con los constructores **con** argumentos, que pueden ser tantos como quieras mientras las listas de parámetros sean distintas:

```
Carrera c1 = new Carrera();           // vacío: se llena después con
Carrera c2 = new Carrera("ISC");      // solo el nombre
Carrera c3 = new Carrera("ISC", "La carrera de ISC"); // todo de golpe
```

– La trampa del constructor que Java te regala

Si **no escribes ningún** constructor, Java te crea uno vacío gratis, y por eso `new Carrera()` funciona aunque no lo hayas puesto.

Pero en cuanto escribes **cualquier** constructor con argumentos, el regalo desaparece. Si escribes solo `Carrera(String nombre)`, entonces `new Carrera()` deja de compilar hasta que declares también el vacío a mano.

Y es justo lo que rompería el DAO: el método `obtenerResultados()` hace `new Carrera()` por cada fila que encuentra. Sin constructor vacío, todo el `readAll()` se cae.

MARCADOR VERDE

BLOQUE 4 · MINUTO 25:34 A 28:28

update()

"Necesito colorcitos." A partir de aquí cambió de marcador por cada método, y por eso esta página está dividida así.

Su descripción fue literalmente "lo mismo": obtiene la conexión, genera un `PreparedStatement`, recibe una `Carrera`. Solo cambian tres cosas.

```
public void update(Carrera c) throws SQLException {  
  
    obtenerConexion();  
    PreparedStatement ps = null;  
  
    try {  
        ps = conexion.prepareStatement( SQL_UPDATE );           // cambio 1: la constante  
        ps.setString(1, c.getNombreCarrera());  
        ps.setString(2, c.getDescripcionCarrera());  
        ps.setInt(3, c.getIdCarrera());                        // cambio 2: un tercer par  
        ps.executeUpdate();  
    } finally {  
        if (ps != null) ps.close();  
        if (conexion != null) conexion.close();  
    }  
}
```

El tercer parámetro es el que trae la novedad, y trae las dos cosas juntas: **índice 3** porque es el tercer `?` de la consulta, y `setInt` porque `idCarrera` es un entero, no texto. Los dos primeros son copiar y pegar del `create`.

La pregunta que salió: ¿dónde se manda el nombre nuevo?

Un alumno preguntó algo que suena básico y no lo es: "*¿en qué parte le mandamos, por ejemplo, el nuevo nombre?*". Es la confusión natural, porque en el método no aparece por ningún lado un "nombre nuevo".

La respuesta es que el método **no distingue** entre viejo y nuevo. El objeto `Carrera` que le llega ya trae los valores que van a quedar. Quien lo llama es el que decide:

```
Carrera c1 = new Carrera();  
c1.setIdCarrera(1); // A CUÁL registro le pega  
c1.setNombreCarrera("LCD"); // los valores NUEVOS  
c1.setDescripcionCarrera("La carrera de LCD");  
  
dao.update(c1);
```

Ese objeto lleva dos cosas de naturaleza distinta y conviene no revolverlas: el **id** dice *a cuál fila* pegarle, y los otros dos campos dicen *con qué valores* dejarla.

– Ojo con esto: es una actualización completa

Él lo dijo de pasada y es importante: "*ya tenemos un método que permite actualizar valores, pero **hace una actualización completa***".

El `SET` reescribe **las dos columnas siempre**. Si solo querías cambiar la descripción y mandas el objeto con `nombreCarrera` en `null`, no es que el nombre se quede como estaba: **se borra** y queda en `null`.

Por eso el flujo correcto es: primero *lee* el registro completo con `read()`, luego cambias en el objeto solo lo que quieres cambiar, y ese objeto completo se lo mandas al `update()`. Es exactamente la razón por la que en el bloque siguiente insiste en que actualizar **incluye** buscar.

MARCADOR ROJO

BLOQUE 5 · MINUTO 30:16 A 32:58

delete()

El más corto de los cinco, y donde se ve mejor que el índice del parámetro no se hereda: se recalcula por consulta.

Borró del pizarrón las dos primeras líneas del `update` porque el `DELETE` solo necesita el id, y luego reescribió la tercera. Ese fue el punto:

"El único detalle: el índice no es el 3, el índice es único, entonces sería 1."


```

public void delete(Carrera c) throws SQLException {

    obtenerConexion();
    PreparedStatement ps = null;

    try {
        ps = conexion.prepareStatement(SQL_DELETE);
        ps.setInt(1, c.getIdCarrera()); // 1, no 3: en ESTA consulta es el único
        ps.executeUpdate(); // DELETE también es executeUpdate
    } finally {
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

```

El índice cuenta los huecos de la consulta que estás ejecutando, no la posición del atributo en la clase ni el orden de las columnas en la tabla. En `SQL_UPDATE` el id es el tercer `?`; en `SQL_DELETE` es el único, así que es el 1.

EJERCICIO 6

Caza los errores en este delete()

Alguien escribió el `delete()` con **cinco errores**. Todos son de los que salieron en clase o de los que él previno explícitamente. **Tócalos para marcarlos.**

0 de 5 encontrados

```

public void delete(Carrera c) throws SQLException {

    obtenerConexion();
    PreparedStatement ps = null;

    try {
        ps = conexion.prepareStatement( SQL_SELECT );
        ps.setInt( c.getIdCarrera() );
        ps.executeQuery();
    } catch (SQLException e) { }
    finally {
        ps.close();
        if (conexion != null) conexion.close();
    }
}

```

"¿Cómo sabe Carrera cuál es su ID?"

La mejor pregunta de la clase, y él mismo la calificó de "muy inocente" en el buen sentido: la respuesta cambia cómo diseñas la aplicación entera.

El planteamiento del alumno: si el `idCarrera` es `auto_increment` y lo genera la base de datos, ¿cómo va a saber tu objeto `Carrera` cuál id le tocó, para poder actualizarlo o borrarlo después?

"La lógica dice esta: lo primero que tienes que hacer es buscar lo que vas a borrar."

Y lo bajó a un ejemplo de pantalla. Imagínate una tabla con un renglón por registro:

ID	NOMBRE	ACCIONES
1	Pedro Picapiedra	ver · actualizar · eliminar
2	Pablo Mármol	ver · actualizar · eliminar
N	...	ver · actualizar · eliminar

Su pregunta al grupo: *"¿cómo sabes tú que si le picas a esto va a actualizar a Pedro Picapiedra?"*. La respuesta que dio un alumno es la correcta: **por el identificador**. Cada botón de cada renglón lleva pegado el id de *su* registro. El id viaja con el botón, no lo adivina nadie.

— ¿Qué es `auto_increment` y por qué complica esto?

Es una propiedad de columna en MySQL: cada vez que insertas una fila sin darle valor a esa columna, la base le pone el siguiente número disponible ella sola.

Por eso el `SQL_INSERT` no menciona `idCarrera`: si se lo mandaras, estarías peleando con la base por quién decide. El objeto `Carrera` que le pasas al `create()` tiene su `idCarrera` en 0 y así se queda, porque el número real se genera *del otro lado* y ese objeto ya no se entera.

De ahí la pregunta del alumno. Y de ahí la respuesta: si necesitas el id, tienes que **volver a consultarlo**. (JDBC tiene además un mecanismo para pedir la clave recién generada, `getGeneratedKeys()`, pero eso no se vio en clase.)

Y esto tiene nombre en Ingeniería de Software: `<<include>>`

Amarró la respuesta con los diagramas de casos de uso: *"el actualizar forzosamente tiene que tener un `<<include>>`, y el `<<include>>` es buscar el registro que quieren actualizar"*.

– Caso de uso y `<<include>>` , explicados desde cero

Un **caso de uso** es una cosa que un usuario quiere lograr con el sistema, escrita en verbo: "Actualizar carrera", "Registrar alumno". En UML se dibuja como un óvalo.

`<<include>>` es una relación entre dos casos de uso que significa **"este otro siempre pasa como parte de este"**. No es opcional, no es un caso especial: siempre ocurre.

Aquí: *Actualizar carrera* `<<include>>` *Buscar carrera*. No hay forma de actualizar sin buscar primero, así que la relación es de inclusión y no una alternativa.

Su ejemplo de la escuela para que quedara claro, y es bueno: el caso de uso es *presentar el protocolo de TT*, y el caso incluido es *entregar el reporte 15 días antes*. Si no entregas el reporte, no presentas. No es opcional: está incluido.

(Sobre la dirección de la flecha él mismo dijo "ahí ustedes me recordarán si es para allá o para allá". Va del caso base al incluido: de *Actualizar* apunta hacia *Buscar*.)

El happy path del DELETE

Más adelante (minuto 43:38) volvió al mismo tema desde el SQL, y ahí soltó el término:

"El cachito del `WHERE` funciona como 've y busca a alguien que tenga el ID tal y elimínalo'. Solo que funciona **muy happy path**. Si no encuentra el ID, no hace nada. Tendría que informarles."

– ¿Qué es el "happy path"?

El **happy path** (o camino feliz) es el recorrido en el que todo sale bien: los datos existen, el usuario escribió bien, la red respondió. Es el caso que uno programa primero y a veces el único que programa.

El problema del `DELETE ... WHERE idCarrera = 99` cuando el 99 no existe es que **no falla**. No lanza excepción, no avisa nada. Simplemente afecta cero filas y se va tan tranquilo. Tu programa cree que borró.

Dos formas de resolverlo, y las dos son válidas:

- La que él propuso: hacer un `SELECT` antes para confirmar que existe, y luego borrar. Es de hecho el mismo `<<include>>` de arriba.
- Aprovechar que `executeUpdate()` **devuelve el número de filas afectadas**. Si te devuelve 0, no había nada que borrar. En el pizarrón se ignoró ese valor de retorno, pero está ahí gratis.

BLOQUE 7 · MINUTO 39:18 A 44:46

La clase Principal: quién usa el DAO

Un alumno dijo que no ubicaba dónde encajaban las piezas ("este sería el DAO, el main, y la parte de allá no veo dónde está"). Entonces borró medio pizarrón y escribió la clase que las junta.

```
// si Carrera y CarreraDAO están en otro paquete, hay que importarlos
import com.ipn.mx.modelo.Carrera;
import com.ipn.mx.dao.CarreraDAO;

public class Principal {

    public static void main(String[] args) {

        // 1. se arma el objeto con los datos
        Carrera c = new Carrera();
        c.setNombreCarrera("LCD");
        c.setDescripcionCarrera("Licenciatura en Ciencia de Datos");

        // 2. se crea el DAO
        CarreraDAO dao = new CarreraDAO();

        // 3. se usa, y AQUÍ sí va el try/catch
        try {
            dao.create(c);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Tres clases, tres trabajos, y ninguna hace el del otro: `Carrera` guarda los datos, `CarreraDAO` habla con la base, `Principal` decide qué se hace. Esa es toda la arquitectura por capas en miniatura.

— `public static void main(String[] args)` : qué significa cada palabra

Es la puerta de entrada: cuando corres el programa, Java busca exactamente este método y empieza ahí.

- `public` — Java tiene que poder llamarlo desde fuera de tu clase.
- `static` — se ejecuta **sin necesidad de crear un objeto** de la clase. Tiene que ser así, porque al arrancar todavía no existe ningún objeto.
- `void` — no devuelve nada.
- `main` — el nombre es obligatorio, Java busca ese exacto.
- `String[] args` — los argumentos que le pasarías desde la línea de comandos. Casi nunca se usan, pero la firma los pide.

Él comentó al pasar que *"para la versión 25 y 26 de Java ese ya no es la firma del main, ya es directo"*. Es cierto: Java moderno permite un `void main()` simplificado. Pero

escribió la larga a propósito, y esa es la que vas a ver en el 99% del código existente y en el examen.

– Paquetes e imports: por qué esas dos líneas de arriba

Un **paquete** es una carpeta con nombre largo que agrupa clases. `com.ipn.mx.modelo` es una carpeta real: `com/ipn/mx/modelo/`.

La regla es simple: si tus clases están **en el mismo paquete**, se ven entre ellas sin hacer nada —"si está en el mismo paquete, pues va a funcionar de lo lindo"—. Si están en paquetes distintos, hay que importarlas.

Fíjate en los nombres que usó, porque describen la separación de responsabilidades: `.modelo` para las clases de datos, `.dao` para las de acceso a base. Esa convención la vas a volver a ver en Spring, con `.controlador`, `.servicio` y `.repositorio`.

Y luego mostró cómo se van rotando las llamadas: para probar cada método, comentas los otros. Para el `delete` hay que agregar el id, que es donde se conecta con la pregunta del bloque anterior:

```
// dao.create(c);
// dao.update(c);
c.setIdCarrera(1);           // aquí es donde decides QUÉ vas a borrar
dao.delete(c);
```

■ MARCADOR NEGRO

BLOQUE 8 • MINUTO 45:13 A 56:35

readAll() y el método que arma la lista

Aquí sí cambia la plantilla. Los tres anteriores no devolvían nada; este devuelve datos, y eso obliga a dos piezas nuevas: un `ResultSet` y un método auxiliar.

```

public List readAll() throws SQLException {

    obtenerConexion();
    PreparedStatement ps = null;
    ResultSet rs = null;           // pieza nueva 1
    List resultados = null;

    try {
        ps = conexion.prepareStatement(SQL_SELECT_ALL);
        // esta consulta NO tiene ?, así que no hay ningún ps.set...
        rs = ps.executeQuery();     // executeQuery, porque es SELECT

        resultados = obtenerResultados(rs); // pieza nueva 2

        if (resultados.size() > 0) {
            return resultados;
        } else {
            return null;
        }
    } finally {
        if (rs != null) rs.close();      // ahora son TRES cierres
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

```

Tres detalles que cambian respecto a los métodos anteriores:

- **No recibe argumentos.** Su explicación: *"¿qué argumentos necesita? Ninguno. Porque la consulta es un SELECT * FROM lo que sea, punto. No hay parámetros"*. Cero `?`, cero `set`.
- **Devuelve una lista.** Sobre el tipo dijo *"ustedes determinen si es una lista tipada o no"* y la dejó genérica. En tu código conviene `List<Carrera>`, que es la versión tipada.
- **Ahora se cierran tres cosas**, y en orden inverso al de apertura: `rs`, `ps`, `conexion`.

¿Qué es un `ResultSet` ?

Es lo que devuelve un `SELECT`: las filas que encontró. Pero **no es una lista** y ahí está la trampa.

Un `ResultSet` es un **cursor**: un dedo apuntando a una fila a la vez, que solo sabe avanzar. Arranca *antes* de la primera fila, y cada `rs.next()` lo mueve una posición y devuelve `true` si cayó en una fila real, o `false` si ya se acabaron. Por eso el recorrido siempre se escribe igual:

```
while (rs.next()) {
    // aquí rs apunta a UNA fila; se leen sus columnas
}
```

Y por eso hay que convertirlo a una lista de objetos: el `ResultSet` **deja de servir en cuanto cierras la conexión**. Si lo devolvieras tal cual desde el DAO, quien lo recibe tendría un objeto muerto. La lista de `Carrera` sí sobrevive porque son objetos de Java normales.

Y de paso, devolver un `ResultSet` rompería el patrón DAO: el resto de la aplicación se enteraría de que existe JDBC.

— ¿Por qué `List` y qué es `List<Carrera>` ?

`List` es una colección ordenada que crece sola, a diferencia de un arreglo, que nace con tamaño fijo. Como no sabes cuántas carreras hay en la tabla, un arreglo no sirve.

`List` es una **interfaz** (dice qué se puede hacer) y `ArrayList` es la **implementación** (dice cómo). Por eso se escribe `List r = new ArrayList();` : declaras por la interfaz, instancias con la implementación. Así puedes cambiar a otra implementación después tocando una sola línea.

Los `<Carrera>` son **genéricos**: le dicen al compilador que ahí solo caben objetos `Carrera`. Sin ellos, la lista acepta cualquier cosa y al sacar elementos tienes que hacer *cast* a mano, con riesgo de reventar en tiempo de ejecución. Con genéricos, el error salta al compilar.

```
List sin = new ArrayList();           // cabe lo que sea
Carrera c = (Carrera) sin.get(0);     // cast obligatorio, puede tronar

List<Carrera> con = new ArrayList<>(); // solo Carreras
Carrera c2 = con.get(0);              // sin cast, seguro
```

Él cerró diciendo *"ustedes lo pueden hacer con lo que quieran: tienen Set, Vector, Collection, lo que se les ocurra"*. Cierto, pero para esto `List` es la correcta: mantiene el orden y admite repetidos.

obtenerResultados(): donde las filas se vuelven objetos

Cuando llegó a la línea de `obtenerResultados(rs)` hizo algo que vale la pena copiarle como técnica:

"¿Qué hace obtenerResultados()? No sé. En este punto no me interesa, solo sé que me regresa una lista y se la asigna a resultados."

Escribió el método de arriba como si el de abajo ya existiera, y hasta después lo escribió. Es la forma sana de atacar un problema: resuelves un

nivel a la vez y confías en que el de abajo cumplirá su contrato.

```
private List obtenerResultados(ResultSet rs) throws SQLException {  
  
    List r = new ArrayList();  
  
    while (rs.next()) {                                // mientras haya fila  
        Carrera c = new Carrera();                    // una instancia POR  
        c.setIdCarrera(rs.getInt("idCarrera"));        // getInt: es entero  
        c.setNombreCarrera(rs.getString("nombreCarrera")); // getString: es VARCHAR  
        c.setDescripcionCarrera(rs.getString("descripcionCarrera"));  
        r.add(c);                                       // se agrega a la lista  
    }  
    return r;  
}
```

Es **privado** porque nadie fuera del DAO lo necesita: es plomería interna. Y fíjate en la simetría con lo que ya sabes: al escribir usabas `ps.setString(1, valor)` con índice numérico; al leer usas `rs.getString("nombreCarrera")` con el **nombre de la columna** entre comillas. Mismos tipos, misma lógica, dirección contraria.

– **Cuidado: ese nombre entre comillas es de la tabla, no de Java**

`rs.getString("nombreCarrera")` le pregunta a la fila por su columna llamada `nombreCarrera`. Ese texto tiene que coincidir con el **nombre de la columna en la base de datos**, no con el nombre del atributo en tu clase.

Aquí coinciden porque las nombraron igual a propósito. El día que la columna se llame `nombre_carrera` y el atributo `nombreCarrera`, el que manda es el de la base.

Y como es un texto entre comillas, **el compilador no lo revisa**: si te equivocas de nombre, Java compila feliz y truena en tiempo de ejecución con un `SQLException` diciendo que esa columna no existe. Es de los errores más comunes en JDBC.

EJERCICIO 7

Arma el método readAll() en orden

Las nueve líneas están revueltas. **Tócalas en el orden correcto** para construir el método. Si te equivocas te aviso de inmediato; toca una línea ya colocada para quitarla.

Aquí se va armando el método...

ResultSet rs = null;

if (rs != null) rs.close();

PreparedStatement ps = null;

rs = ps.executeQuery();

return obtenerResultados(rs);

try {

} finally {

obtenerConexion();

ps = conexion.prepareStatement(SQL_SELECT_ALL);

Reiniciar

read(): el mismo camino, pero devolviendo uno solo

El quinto y último. *"No necesito una lista. ¿Qué hace el SELECT con WHERE? Me devuelve un registro. Un registro es una instancia de Carrera."*

```

public Carrera read(Carrera c) throws SQLException {    // devuelve Carrera, no Lis

    obtenerConexion();
    PreparedStatement ps = null;
    ResultSet rs = null;
    List resultados = null;

    try {
        ps = conexion.prepareStatement(SQL_SELECT);    // el que lleva WHERE
        ps.setInt(1, c.getIdCarrera());                // este SÍ tiene un ?
        rs = ps.executeQuery();

        resultados = obtenerResultados(rs);            // el mismo método auxiliar

        if (resultados.size() > 0) {
            return (Carrera) resultados.get(0);        // la primera coincidencia
        } else {
            return null;
        }
    } finally {
        if (rs != null) rs.close();
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

```

Reusa `obtenerResultados()` tal cual, y el truco está en la última parte: como la consulta filtra por clave primaria, la lista va a traer un solo elemento. `resultados.get(0)` lo saca. **Aquí sí es cero**, porque `get(0)` es un índice de lista de Java, no un parámetro de SQL. Los dos conviven en el mismo método y no se contradicen.

— Los dos "índices" del método, para que no se te revuelvan

En este método aparecen un 1 y un 0 muy juntos y significan cosas distintas:

- `ps.setInt(1, ...)` — el **primer hueco de la consulta SQL**. Los parámetros de JDBC empiezan en 1.
- `resultados.get(0)` — el **primer elemento de una lista de Java**. Las colecciones de Java empiezan en 0.

Regla para no dudar: **si el número se lo estás dando a SQL, empieza en 1. Si se lo estás dando a Java, empieza en 0.**

Reporte de lectura — "Lectura rápida JDBC"

"Pregunta inocente de nuevo: ¿ya hicieron su lectura rápida de JDBC? Se los voy a pedir de tarea."

No es nueva, es la misma que ya venía arrastrándose. Ya la había dejado como reporte de lectura en la clase 3. Aquí volvió a preguntarla, nadie contestó que sí, y la confirmó. Si ya la tenías pendiente, sigue pendiente.

Y sí importa para esta clase: el `CallableStatement` del último ejercicio sale justo de esa lectura. Él lo dijo así: *"según su lectura rápida JDBC, el otro es este cuate"*. O sea que el ejercicio da por hecho que ya la leíste.

EJERCICIO DEL JEFE DE GRUPO · MINUTO 63:35 A 67:00

"Hoy ando en modo tareas"

Lo dijo él mismo antes de escribir las especificaciones en el pizarrón. Y sí: la reacción del grupo al oír "interfaz gráfica para Windows" fue un "¡Nooo!" colectivo.

Ejercicio 1 — Aplicación de escritorio con interfaz gráfica

"Interfaz gráfica de usuario con botoncitos para Windows. Aplicación de escritorio para darle soporte a la carrera. Así, bien vintage. Échenle a volar la imaginación."

Qué significa "para Windows". Alguien preguntó si podía ser en otra cosa y aclaró que era genérico: *"al decir para Windows es el genérico, o sea, es con ventanitas y botones, nada más. Interfaz gráfica de usuario basada en Swing, en JavaFX o lo que quieran. No web, no móvil."*

Qué tiene que hacer. Darle soporte al CRUD de `Carrera` que ya escribieron. La lógica ya está en `CarreraDAO`: esto solo le pone cara. Donde en `Principal` comentabas y descomentabas líneas, ahora hay botones.

El flujo que describió, con el detalle de habilitar y deshabilitar controles, que es la parte que quiere ver:

- Al cargar, el único botón activo es **Nuevo**. Todo lo demás, bloqueado.
- Le picas a **Nuevo**: se oculta el campo de `idCarrera` (no lo necesitas, lo genera la base), se habilitan nombre y descripción, y aparece **Guardar**. Un **Cancelar** si quieres.
- Llenas, le picas a **Guardar**, el registro se va a la tabla, y todo se deshabilita otra vez: vuelve al estado inicial.
- **Buscar** hace lo simétrico: habilita unos campos, oculta otros.

Sin traumas. Lo dijo dos veces: *"nada más no se traumen con eso, simplemente que tenga interfaz gráfica de usuario. Nosotros no vamos a hacer cosas de escritorio, pero simplemente se puede"*. No es el proyecto del semestre; es para que veas que el DAO no depende de quién lo llame.

– Swing y JavaFX: cuál es cuál

Las dos son librerías de Java para hacer ventanas de escritorio. Ninguna tiene que ver con web.

- **Swing** — la vieja, de finales de los 90. Viene incluida en el JDK, así que no instalas nada. Fea de fábrica pero funcional, y hay ejemplos por todos lados. Es la más rápida para salir del paso.
- **JavaFX** — la moderna. Se ve mucho mejor, permite estilizar con CSS y diseñar la ventana en un archivo aparte (FXML). Ya no viene en el JDK: hay que agregarla como dependencia.

Para esta tarea, Swing te va a costar menos. Si tu IDE tiene diseñador visual de formularios (NetBeans lo trae para Swing), arrastras los botones y ya.

El concepto que sí importa que entiendas es **habilitar y deshabilitar**: cada control tiene un método (`setEnabled(true/false)`) que lo prende o lo apaga, y otro (`setVisible()`) que lo muestra u oculta. Todo el flujo que describió es prender y apagar controles según

en qué momento va el usuario. Deshabilitar es una forma de evitar errores: si el botón no se puede picar, no hay nada que validar después.

PENDIENTE ADMINISTRATIVO minuto 28:52

Cursos externos: subir tus datos al Excel

Un alumno reportó que al registrarse en el curso de Red Hat les manda error. La aclaración: **los cursos todavía no están creados**, por eso falla.

Lo que él necesita de ustedes, en el Excel compartido: **usuario, correo electrónico y qué curso quieren**. Con eso los da de alta.

Recuerda por qué conviene: al inicio de clase mencionó que son tres exámenes, y con **85 o más en los tres** quedas como candidato al examen de certificación. El catálogo completo está disponible; el curso que pidan, se abre.

BLOQUE 9 · MINUTO 67:49 A 71:34

Stored procedures: la misma inserción, del lado de la base

"¿A quién le gusta ser DBA aquí? A ninguno de ustedes les gustan las bases de datos, ¿no? ¡Qué bárbaro!" Y con eso arrancó la última parte, que existe para preparar el ejercicio 2.

— ¿Qué es un stored procedure? ¿Y un DBA?

Un **stored procedure** (procedimiento almacenado) es código SQL con nombre que vive **guardado dentro de la base de datos**, no en tu aplicación. Lo escribes una vez, se queda ahí, y desde Java solo lo mandas llamar por su nombre.

Es como un método, pero del lado de la base: recibe parámetros, ejecuta instrucciones y puede devolver resultados.

Ventajas: la lógica queda en un solo lugar aunque la usen varias aplicaciones, viaja menos texto por la red, y la base puede tenerlo pre-optimizado. Desventajas: queda amarrado a ese manejador, es más difícil de versionar en Git, y te reparte la lógica de negocio entre dos lugares.

Un **DBA** (Database Administrator) es la persona encargada de la base de datos: diseño, respaldos, permisos, rendimiento. Los stored procedures suelen ser su territorio, y por eso preguntó a quién le gustaba serlo.

La advertencia antes de escribirlo, que es la que va al examen:

*"Aquí es literal: **depende del manejador de datos**. No es lo mismo esto que vamos a apuntar que va a funcionar en un MySQL que si se lo*

quieren llevar a un PostgreSQL, a un Oracle o lo que sea. Ahí el ANSI SQL dejó de ser tú."

– ¿Qué es "ANSI SQL" y por qué "dejó de ser tú"?

ANSI SQL es el SQL estándar, el que todos los manejadores acordaron soportar igual. Los `SELECT`, `INSERT`, `UPDATE` y `DELETE` básicos que llevas escribiendo toda la clase son ANSI: los mueves de MySQL a PostgreSQL y funcionan sin tocarlos.

Los procedimientos y funciones **no**. Cada manejador se inventó su propio lenguaje: Oracle usa PL/SQL, SQL Server usa T-SQL, PostgreSQL usa PL/pgSQL, MySQL el suyo. Cambia la sintaxis para declarar parámetros, para el cuerpo, y hasta la palabra para invocarlo.

La consecuencia práctica: en cuanto usas stored procedures, **cambiar de base de datos deja de ser gratis**. Y eso choca de frente con la razón de existir del patrón DAO, que era justamente aislar ese cambio.

```
DELIMITER //
```

```
CREATE PROCEDURE sp_crear(nombre VARCHAR(50), descripcion VARCHAR(100))
BEGIN
    INSERT INTO Carrera (nombreCarrera, descripcionCarrera)
    VALUES (nombre, descripcion);
END //

DELIMITER ;
```

Fíjate en que **adentro no hay ni un ?**. Los huecos se volvieron parámetros con nombre (`nombre`, `descripcion`) y se usan directo. Los tipos y tamaños los sacó de cómo esté definida la tabla —*"no me acuerdo qué tamaños tenía esa cosa, depende de cómo esté definido en bases de datos"*—, así que revisa los tuyos.

– El `DELIMITER` : por qué sin él "les va a decir de groserías"

Normalmente el punto y coma le dice a MySQL "aquí termina la instrucción, ejecútala".

El problema es que el cuerpo del procedimiento **tiene punto y comas adentro** (los del `INSERT`). Sin protección, MySQL lee hasta el primer `;`, cree que ahí terminó el `CREATE PROCEDURE`, e intenta ejecutar algo incompleto. De ahí el error.

`DELIMITER //` cambia temporalmente el terminador a `//`. Ahora los `;` de adentro son texto normal, y el procedimiento termina cuando aparece `//`. Al final, `DELIMITER ;` lo regresa a la normalidad para que el resto de tu sesión siga funcionando.

Ese último `DELIMITER ;` se olvida muchísimo, y el síntoma es raro: todo lo que escribas después parece no ejecutarse.

Y una cosa que dejó abierta: *"aquí pueden poner todas las condicionales que quieran, todo el manejo de error que quieran propio del SQL para determinar que lo que están recibiendo realmente sea una cadena de texto válida. Pero eso ya les toca a ustedes"*.

EJERCICIO 8

Escribe la llamada al procedimiento

Ya está creado el `sp_crear`. Desde la línea de comandos de MySQL, **invócalo** pasándole `'LCD'` como nombre y `'LCD'` como descripción, tal como lo escribió él en el pizarrón.

escribe la instrucción completa

Revisar

Limpiar

Un alumno preguntó si no era `EXEC`. La respuesta define bien el punto del bloque entero:

"No, porque no es SQL Server. Eso es para SQL Server, para Oracle también creo. Para MySQL es CALL. Por eso les decía, aquí sí es diferente."

EJERCICIO 9

Lo que cambia entre manejadores

Cuatro preguntas de cierre sobre stored procedures y sobre el detalle que corrigió el compañero atento al final de la clase.

¿Para qué sirve el `DELIMITER //` antes del `CREATE PROCEDURE` ?

Para separar parámetros

Para que los `;` de adentro no corten la instrucción

Para comentar el bloque

Escribiste el procedimiento en MySQL y lo quieres llevar a PostgreSQL. ¿Funciona igual?

Sí, es ANSI SQL

No, el lenguaje de procedimientos cambia por manejador

En MySQL, ¿cómo se invoca un procedimiento almacenado?

EXEC `sp_crear(...)`

CALL `sp_crear(...)`

RUN `sp_crear(...)`

La Tarea 4 pide cambiar a `CallableStatement`. ¿Qué NO cambia?

Los índices de parámetros siguen empezando en 1

Se sigue usando `prepareStatement()`

Ya no hay que cerrar nada

Revisar

Reiniciar

Ejercicio 2 — Cambiar PreparedStatement por CallableStatement

"Esto lo tienen con PreparedStatement, cámbienlo con CallableStatement. Según su lectura rápida JDBC, el otro es este cuate: el CallableStatement. ¿Para cuándo? Para la próxima semana."

Es la única tarea con fecha explícita. Las otras tres no la llevan; esta sí.

Qué implica. Los cinco métodos del DAO se quedan igual en estructura. Lo que cambia es que en vez de preparar SQL suelto, mandas llamar procedimientos almacenados. O sea que primero tienes que crear los procedimientos en MySQL: él solo escribió el de insertar, los otros cuatro te tocan.

La forma general del cambio, para que veas que es menos de lo que parece:

```
// antes
PreparedStatement ps = conexion.prepareStatement(SQL_INSERT);

// después
CallableStatement cs = conexion.prepareCall("{call sp_crear(?, ?)}");
```

Los `?` siguen ahí y los `setString` / `setInt` se usan igual, con la misma numeración desde 1. Cuando señaló qué se iba a cambiar dijo *"ah, pues sí, sí, sí, ese, ese, ese"* apuntando a esas líneas: es un reemplazo bastante mecánico.

Y la razón de que la Tarea 2 importe: `CallableStatement` tiene una capacidad que los otros no —los parámetros de salida (`OUT`)— y eso viene explicado en la lectura rápida de JDBC. Ahí es donde deja de ser mecánico.

REFERENCIA

El DAO completo, de corrido

Todo lo del pizarrón junto y en orden, ya con los imports que hacen falta y con la lista tipada. Esta es la versión contra la que puedes comparar tu archivo.

CarreraDAO.java

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;

public class CarreraDAO {

    private static final String SQL_INSERT =
        "INSERT INTO Carrera (nombreCarrera, descripcionCarrera) VALUES (?, ?)";
    private static final String SQL_UPDATE =
        "UPDATE Carrera SET nombreCarrera = ?, descripcionCarrera = ? WHERE idCarrera = ";
    private static final String SQL_DELETE =
        "DELETE FROM Carrera WHERE idCarrera = ?";
    private static final String SQL_SELECT =
        "SELECT * FROM Carrera WHERE idCarrera = ?";
    private static final String SQL_SELECT_ALL =
        "SELECT * FROM Carrera";

    private Connection conexion;

    private void obtenerConexion() throws SQLException, ClassNotFoundException {
        String usuario = "root";
        String clave = "tu_password";
        String urlBD = "jdbc:mysql://localhost:3306/MisCursos";
        String driverMySQL = "com.mysql.cj.jdbc.Driver";

        Class.forName(driverMySQL);
        conexion = DriverManager.getConnection(urlBD, usuario, clave);
    }

    // ----- create -----
    public void create(Carrera c) throws SQLException {
        obtenerConexion();
        PreparedStatement ps = null;
        try {
            ps = conexion.prepareStatement(SQL_INSERT);
            ps.setString(1, c.getNombreCarrera());
            ps.setString(2, c.getDescripcionCarrera());
            ps.executeUpdate();
        } finally {
            if (ps != null) ps.close();
            if (conexion != null) conexion.close();
        }
    }

    // ----- update -----

```

```

public void update(Carrera c) throws SQLException {
    obtenerConexion();
    PreparedStatement ps = null;
    try {
        ps = conexion.prepareStatement(SQL_UPDATE);
        ps.setString(1, c.getNombreCarrera());
        ps.setString(2, c.getDescripcionCarrera());
        ps.setInt(3, c.getIdCarrera());
        ps.executeUpdate();
    } finally {
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

// ----- delete -----
public void delete(Carrera c) throws SQLException {
    obtenerConexion();
    PreparedStatement ps = null;
    try {
        ps = conexion.prepareStatement(SQL_DELETE);
        ps.setInt(1, c.getIdCarrera());
        ps.executeUpdate();
    } finally {
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

// ----- readAll -----
public List<Carrera> readAll() throws SQLException {
    obtenerConexion();
    PreparedStatement ps = null;
    ResultSet rs = null;
    List<Carrera> resultados = null;
    try {
        ps = conexion.prepareStatement(SQL_SELECT_ALL);
        rs = ps.executeQuery();
        resultados = obtenerResultados(rs);
        if (resultados.size() > 0) {
            return resultados;
        } else {
            return null;
        }
    } finally {
        if (rs != null) rs.close();
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

```

```
// ----- read -----
public Carrera read(Carrera c) throws SQLException {
    obtenerConexion();
    PreparedStatement ps = null;
    ResultSet rs = null;
    List<Carrera> resultados = null;
    try {
        ps = conexion.prepareStatement(SQL_SELECT);
        ps.setInt(1, c.getIdCarrera());
        rs = ps.executeQuery();
        resultados = obtenerResultados(rs);
        if (resultados.size() > 0) {
            return resultados.get(0);
        } else {
            return null;
        }
    } finally {
        if (rs != null) rs.close();
        if (ps != null) ps.close();
        if (conexion != null) conexion.close();
    }
}

// ----- auxiliar -----
private List<Carrera> obtenerResultados(ResultSet rs) throws SQLException {
    List<Carrera> r = new ArrayList<>();
    while (rs.next()) {
        Carrera c = new Carrera();
        c.setIdCarrera(rs.getInt("idCarrera"));
        c.setNombreCarrera(rs.getString("nombreCarrera"));
        c.setDescripcionCarrera(rs.getString("descripcionCarrera"));
        r.add(c);
    }
    return r;
}
}
```

– Tres diferencias entre esta versión y lo que quedó en el pizarrón

No son correcciones a lo que él dijo: son los cabos que dejó sueltos a propósito por tiempo, y conviene que sepas cuáles son.

- **Las listas van tipadas** (`List<Carrera>`). Él las dejó genéricas y dijo explícitamente *"ustedes determinen si es una lista tipada o no"*. Tipadas te ahorran los *casts* y los errores en tiempo de ejecución.
- `obtenerConexion()` **declara sus excepciones** en vez de atraparlas con `printStackTrace()`. Es coherente con la estafeta que él mismo explicó: si la conexión falla, el método no debería seguir como si nada.

- `read()` usa `SQL_SELECT`, el que lleva `WHERE`. Es fácil dejar pegado el `SQL_SELECT_ALL` al copiar el método de arriba, y el síntoma es feo: te devuelve la primera carrera de la tabla en vez de la que pediste, sin lanzar ningún error.

Sobre lo que sí quedó pendiente de verdad: el `create()` no recupera el id que generó la base, y el `delete()` ignora el número de filas afectadas. Los dos temas salieron en clase pero no se resolvieron en código.

CIERRE

Las cuatro tareas, juntas

Cerró seis minutos antes, después de que un alumno le cachara el índice faltante en un `setInt`. Aquí están las cuatro tareas más el pendiente de los cursos, para ir marcándolas.

0 / 5

☐ Tarea 1 · Investigación: soft delete

Qué es, para qué sirve, con qué se come, cómo se adereza y cómo lo aplican. Borrado lógico contra borrado físico. Es para el proyecto, no solo para entregar.

☐ Tarea 2 · Reporte de lectura: "Lectura rápida JDBC"

Venía arrastrándose desde la clase 3 y aquí la reconfirmó. El `CallableStatement` de la Tarea 4 sale de esta lectura, así que va primero.

☐ Tarea 3 · Ejercicio 1: aplicación de escritorio con interfaz gráfica

Swing o JavaFX, no web ni móvil. CRUD de Carrera con botones que se habilitan y deshabilitan según el momento. Reusa el `CarreraDAO` tal cual.

☐ Tarea 4 · Ejercicio 2: cambiar `PreparedStatement` por `CallableStatement`

La única con fecha: para la próxima semana. Implica crear antes los cinco stored procedures en MySQL; él solo escribió el de insertar.

☐ Pendiente · Subir tus datos para el curso externo

Usuario, correo y qué curso quieres, en el Excel. Los cursos aún no están creados, por eso marca error al registrarse. Da punto extra en el examen.

Cuál urge. La única con fecha es el **Ejercicio 2** (`CallableStatement`), para la próxima semana. Pero depende de la **Tarea 2** (lectura de JDBC) y de que tengas los procedimientos creados en MySQL, así que en la práctica el orden es: leer JDBC, escribir los cinco stored procedures, y hasta entonces cambiar el DAO.

La **Tarea 1** (soft delete) es investigación corta y va con el formato de siempre. La **Tarea 3** (interfaz gráfica) es la más larga en horas, pero él insistió en que no se traumen: es para comprobar que el DAO funciona igual sin importar quién lo llame.